

**МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ
КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ**

**Ордена трудового Красного Знамени федеральное государственное
бюджетное образовательное учреждение высшего образования**

«Московский технический университет связи и информатики»

Кафедра Математическая кибернетика и информационные технологии

Лабораторная работа №7.

Выполнил: студент группы БПИ2401

Исламов Эмин Маратович

Проверил: Харрасов Камиль Раисович

Цель работы

Изучить основы многопоточности в Java: создание потоков через наследование Thread, запуск параллельных вычислений, синхронизация результатов в главном потоке с помощью join(), а также моделирование конкурентного доступа к общему ресурсу.

Ход работы

1. Реализовать задачу 1: два потока считают сумму половин массива, затем главный поток складывает результаты.
2. Реализовать задачу 2: несколько потоков ищут максимум по строкам матрицы, главный поток выбирает общий максимум.
3. Реализовать задачу 3: модель склада с товарами (весами) и тремя грузчиками-потоками; за одну “поездку” каждый грузчик переносит не больше 150 кг; перенос продолжается, пока склад не опустеет.
4. Проверить корректность работы программ на тестовых данных.

Задание 1

Формулировка

Два потока вычисляют сумму элементов массива по половинкам, результаты складываются в главном потоке.

```
import java.util.*;

public class SumThreads {

    static class SumThread extends Thread {
        private final int[] a;
        private final int l, r; // [l, r)
        private long sum;

        SumThread(int[] a, int l, int r) {
            this.a = a;
            this.l = l;
            this.r = r;
        }
    }
}
```

```

    }

    @Override
    public void run() {
        long s = 0;
        for (int i = l; i < r; i++) s += a[i];
        sum = s;
    }

    public long getSum() {
        return sum;
    }
}

public static void main(String[] args) throws Exception {
    Scanner sc = new Scanner(System.in);

    int n = sc.nextInt();
    int[] a = new int[n];
    for (int i = 0; i < n; i++) a[i] = sc.nextInt();

    int mid = n / 2;

    SumThread t1 = new SumThread(a, 0, mid);
    SumThread t2 = new SumThread(a, mid, n);

    t1.start();
    t2.start();

    t1.join();
    t2.join();

    long total = t1.getSum() + t2.getSum();
    System.out.println(total);
}
}

```

Объяснение кода

- Создаём класс SumThread extends Thread, который получает массив и границы отрезка [l, r); в run() считается сумма на своём отрезке.
- В main() делим массив на две части, запускаем два потока, а затем **ждём их завершения** через join(), чтобы значения сумм точно были посчитаны.
- После join() складываем результаты и печатаем.

Задание 2

Формулировка

Создать несколько потоков, каждый обрабатывает **свою строку** матрицы и ищет максимум в этой строке, затем главный поток выбирает общий максимум.

Код (MatrixMaxThreads.java)

```
import java.util.*;

public class MatrixMaxThreads {

    static class RowMaxThread extends Thread {
        private final int[][] m;
        private final int row;
        private int rowMax;

        RowMaxThread(int[][] m, int row) {
            this.m = m;
            this.row = row;
        }

        @Override
        public void run() {
            int mx = m[row][0];
            for (int j = 1; j < m[row].length; j++) {
                if (m[row][j] > mx) mx = m[row][j];
            }
            rowMax = mx;
        }

        public int getRowMax() {
            return rowMax;
        }
    }

    public static void main(String[] args) throws Exception {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int k = sc.nextInt();
        int[][] m = new int[n][k];

        for (int i = 0; i < n; i++)
            for (int j = 0; j < k; j++)
```

```

        m[i][j] = sc.nextInt();

        RowMaxThread[] threads = new RowMaxThread[n];
        for (int i = 0; i < n; i++) {
            threads[i] = new RowMaxThread(m, i);
            threads[i].start();
        }

        for (int i = 0; i < n; i++) threads[i].join();

        int globalMax = threads[0].getRowMax();
        for (int i = 1; i < n; i++) {
            if(threads[i].getRowMax() > globalMax)
                globalMax = threads[i].getRowMax();
        }

        System.out.println(globalMax);
    }
}

```

Объяснение кода

- Для каждой строки создаётся RowMaxThread, который ищет максимум только в одной строке.
- Главный поток запускает все потоки, затем делает join() по каждому, чтобы дождаться результатов.
- После этого берём максимум среди rowMax всех потоков.

Задание 3

Формулировка

Есть склад с товарами (у каждого товара вес). Работают 3 грузчика параллельно. Грузчик за одну итерацию переносит товары суммарным весом **не больше 150 кг**, затем “уезжает” на другой склад и разгружает. Повторять, пока исходный склад не пуст.

Код (WarehouseThreads.java)

```

import java.util.*;

public class WarehouseThreads {

```

```

static class Item {
    final String name;
    final int weight;

    Item(String name, int weight) {
        this.name = name;
        this.weight = weight;
    }
}

static class Warehouse {
    private final Deque<Item> items = new ArrayDeque<>();

    public synchronized void add(Item it) {
        items.addLast(it);
    }

    // выдем партию товаров со склада не более limit, 150
    public synchronized List<Item> takeBatch(int limit) {
        if (items.isEmpty()) return Collections.emptyList();

        List<Item> batch = new ArrayList<>();
        int total = 0;

        // берём по очереди, пока помещается
        while (!items.isEmpty()) {
            Item next = items.peekFirst();
            if (total + next.weight > limit)
                break;
            batch.add(items.removeFirst());
            total += next.weight;
        }
        return batch;
    }

    public synchronized boolean isEmpty() {
        return items.isEmpty();
    }

    public synchronized int count() {
        return items.size();
    }
}

static class Loader extends Thread {
    private final Warehouse from;
    private final Warehouse to;
    private final int limit;

```

```

    Loader(String name, Warehouse from, Warehouse to, int limit) {
        super(name);
        this.from = from;
        this.to = to;
        this.limit = limit;
    }

    @Override
    public void run() {
        while (true) {
            List<Item> batch = from.takeBatch(limit);

            if (batch.isEmpty()) {
                //если склад пуст завершаемся
                if (from.isEmpty()) break;
                break;
            }

            // перенос на другой склад
            for (Item it : batch) {
                to.add(it);
            }
        }
    }
}

public static void main(String[] args) throws Exception {
    Scanner sc = new Scanner(System.in);
    int n = sc.nextInt();//кол-во товаров

    Warehouse w1 = new Warehouse();
    Warehouse w2 = new Warehouse();

    for (int i = 0; i < n; i++) {
        String name = sc.next();
        int weight = sc.nextInt();
        w1.add(new Item(name, weight));
    }

    int limit = 150;

    Loader l1 = new Loader("грузчик 1", w1, w2, limit);
    Loader l2 = new Loader("грузчик 2", w1, w2, limit);
    Loader l3 = new Loader("грузчик 3", w1, w2, limit);

    l1.start();
    l2.start();

```

```

        l3.start();

        l1.join();
        l2.join();
        l3.join();

        System.out.println(w2.count());
    }
}

```

Объяснение кода

- Warehouse хранит товары в Deque. Методы add, takeBatch, isEmpty, count сделаны synchronized, чтобы **одновременно** несколько грузчиков не “забрали один и тот же товар”.
- takeBatch(limit) формирует список товаров на одну поездку: вытаскивает из начала очереди, пока суммарный вес не превышает 150.
- Loader extends Thread в цикле берёт партию со склада 1, перекладывает её на склад 2, повторяет, пока склад 1 не опустеет.
- В main() создаём 3 грузчика, запускаем и ждём через join(), потом печатаем итог.

Вывод

В ходе работы были реализованы три многопоточные программы с использованием Thread: параллельный подсчёт суммы массива, параллельный поиск максимума в матрице по строкам и моделирование конкурентного переноса товаров между складами. Для корректного объединения результатов применён join(), а для безопасного доступа к общему складу — синхронизация (synchronized), что предотвращает гонки данных и дублирование выдачи товаров.

Контрольные вопросы — ответы

1. Как реализуется многопоточность в Java?

Через создание и запуск потоков (Thread), либо через задачи Runnable/Callable и пул потоков (ExecutorService), где параллельное выполнение обеспечивается планировщиком ОС и JVM.

2. Что такое поток?

Поток — это отдельная “линия выполнения” внутри процесса, имеющая свой стек вызовов, но разделяющая память кучи с другими потоками того же процесса.

3. Для чего нужно ключевое слово `synchronized`?

Чтобы сделать критическую секцию взаимно-исключающей: в один момент времени только один поток может выполнять синхронизированный метод/блок для данного монитора, и одновременно обеспечивается корректная видимость изменений.

4. Для чего нужно ключевое слово `volatile`?

Чтобы гарантировать, что чтения/записи переменной происходят напрямую из общей памяти и изменения становятся видимыми другим потокам, хотя атомарность сложных операций (например `x++`) `volatile` не даёт.

5. Зачем синхронизировать потоки?

Чтобы избежать гонок данных, некорректных состояний объектов и ошибок при одновременном доступе к общим ресурсам.

6. Какие есть способы синхронизации потоков?

`synchronized`, `volatile`, классы из `java.util.concurrent` (`Lock/Condition`, `Semaphore`, `CountDownLatch`, `CyclicBarrier`), атомарные типы (`AtomicInteger`), очереди и блокировки внутри коллекций.

7. В чем разница между `Thread` и `Runnable`?

`Thread` — это сам поток исполнения, а `Runnable` — это задача (код), которую поток выполняет; один и тот же `Runnable` можно запускать разными потоками, что удобнее для архитектуры.

8. Какие состояния может иметь поток?

`NEW`, `RUNNABLE`, `BLOCKED`, `WAITING`, `TIMED_WAITING`, `TERMINATED`.

9. Что такое daemon-поток и как его создать?

Daemon — служебный поток, который не удерживает JVM от завершения; создаётся через `thread.setDaemon(true)` до `start()`.

10. Как принудительно остановить поток?

Корректно — через флаг остановки/`interrupt()` и проверку `isInterrupted()`/обработку `InterruptedException`; `stop()` устарел и опасен.

11. Как работает `join()` и для чего он используется?

`join()` блокирует текущий поток до завершения указанного потока, поэтому он нужен, чтобы дождаться результата параллельной работы.

12. Что такое “гонка данных” (race condition)?

Ситуация, когда итог зависит от порядка выполнения потоков, потому что доступ к общим данным происходит без правильной синхронизации.

13. Что такое deadlock и как его избежать?

Deadlock — взаимная блокировка потоков при захвате разных ресурсов в разном порядке; избегают фиксированным порядком захвата, таймаутами, `tryLock`, уменьшением количества блокировок.

14. Что такое `wait()`, `notify()`, `notifyAll()` и где объявлены?

Это методы класса `Object` для координации потоков через монитор: `wait()` усыпляет поток и освобождает монитор, `notify()` будит один ожидающий, `notifyAll()` будит всех; используются только внутри `synchronized`.

15. Что такое `ThreadPool`? Какие реализации `ExecutorService` есть?

`ThreadPool` — набор заранее созданных потоков для выполнения задач без постоянного создания/уничтожения потоков; типичные реализации: `newFixedThreadPool`, `newCachedThreadPool`, `newSingleThreadExecutor`, `newScheduledThreadPool`, `ForkJoinPool`.